

Instructions & Operators

Chapter 2 • C Programming Core Fundamentals

INSTRUCTIONS

Instructions are statements written in a program to direct the execution flow.

Type Declaration

Used to declare variable names and lock their memory size and data type before operations begin.

Arithmetic

Performs standard operational values manipulation utilizing operators like addition, division, and subtraction.

Control Flow

Used to intentionally govern and change the linear step-by-step path of program execution.

TYPE DECLARATION

Variables must be explicitly declared with their proper type context prior to practical usage.

VALID Instructions

```
int a = 22;
```

```
int b = a;
```

```
int c = b + 1;
```

```
int d = 1, e;
```

INVALID Instructions

```
int a, b, c = 1;
```

(Leaves variables 'a' and 'b' uninitialized before binding value)

ARITHMETIC FORMATS

Anatomy of an Expression

An assignment expression holds variables/values acting as operands interacting via symbols.

$$a + b$$

LHS Rule: Only a single variable can occupy the Left-Hand Side (LHS) of an assignment operation.

```
// Correct assignment  
int total = a + b;
```

```
// Invalid assignment structure  
a + b = total;
```

ARITHMETIC RULES

VALID Code	INVALID Code	Operational Reason
<code>a = b * c;</code>	<code>a = bc;</code>	The multiplication sign <code>*</code> must be manually declared.
<code>a = b / c;</code>	<code>a = b ^ c;</code>	The <code>^</code> character is a Bitwise XOR, not an exponential power symbol.
<code>int p = pow(b, c);</code>		Always use the built-in <code>pow(x, y)</code> library function for calculations.

MODULAR OPERATOR

The % Symbol

Returns the explicit **remainder value** from an integer division calculation sequence.

Data Constraint: Can only be successfully applied on whole number integer values.

3 % 2

Returns a remainder value of: **1**

-3 % 2

Returns a remainder value of: **-1**

TYPE CONVERSION

Operations mix data types implicitly to match the widest precision format involved.

int • int

Returns `int`



int • float

Returns `float`



float • float

Returns `float`

Example: `5 / 2` evaluates to `2` (integer division). While `5.0 / 2` evaluates to `2.5`.

OPERATOR PRECEDENCE

Precedence dictates execution priority when a single expression references varying symbols.

Priority	Operators
1 (Highest)	<code>*</code> <code>/</code> <code>%</code>
2	<code>+</code> <code>-</code>
3 (Lowest)	<code>=</code>

Evaluation Sample:

$$x = 4 + 9 * 10$$

Step 1: `9 * 10` executes first → `90`

Step 2: `4 + 90` executes second → `x = 94`

ASSOCIATIVITY

Left-to-Right Rules

When neighboring operators hold the **exact same precedence tier**, compliance routes from Left to Right.

$$x = 4 * 3 / 6 * 2$$

Sequence Path Breakdown:

1. Step 1: First operation encounters $4 * 3$ → Evaluates to 12
2. Step 2: Expression morphs into $12 / 6 * 2$. Next operation scales $12 / 6$ → Evaluates to 2
3. Step 3: Remaining statement converts to $2 * 2$ → Final assignment records $x = 4$

CONTROL INSTRUCTIONS

Control statements regulate execution sequence blocks within compilation paths.

Sequence

Default
format style
processing
statements
sequentially
step-by-step.

Decision

Forks
pathway
directions
checking
conditions
via
structures
like `if-`
`else` .

Loop

Repeats statements
continuously utilizing
`while` , `for` or
`do-while`
iterations.

Case

Instantly targets
execution tags
utilizing
`switch-case`
selectors.

OPERATOR SUITE

C splits logical elements into specialized operational groups to classify expression designs:

Arithmetic

Relational

Logical

Bitwise

Assignment

Ternary

RELATIONAL & LOGICAL

Relational Operators

Tests conditional relationships between operands:

`==` Equal To

`!=` Not Equal

`>` Greater Than

`>=` Greater or Equal

`<` Less Than

`<=` Less or Equal

Logical Operators

Connects separate boolean expressions together:

`&&`

AND • True only if both sides evaluate true

`||`

OR • True if at least one side evaluates true

`!`

NOT • Flips the structural condition state output

MASTER PRIORITY LIST

Priority	Operators	Description Classification
1	!	Logical Unary NOT Operator
2	*, /, %	Multiplication, Mathematical Division, and Remainder Modulo
3	+, -	Mathematical Addition and Basic Subtraction
4	<, <=, >, >=	Relational Comparative Evaluations
5	==, !=	Equality and Inequality Structural Checks
6	&&	Logical Conjunction AND
7		Logical Disjunction OR
8	=	Standard Expression Assignment

COMPOUND ASSIGNMENT

Shorthand Symbols

Compound operators merge arithmetic expressions and assignment loops together neatly.

Operator	Sample Syntax	Expanded Equivalent Statement Meaning
<code>+=</code>	<code>x += y</code>	<code>x = x + y</code>
<code>-=</code>	<code>x -= y</code>	<code>x = x - y</code>
<code>*=</code>	<code>x *= y</code>	<code>x = x * y</code>
<code>/=</code>	<code>x /= y</code>	<code>x = x / y</code>
<code>%=</code>	<code>x %= y</code>	<code>x = x % y</code>